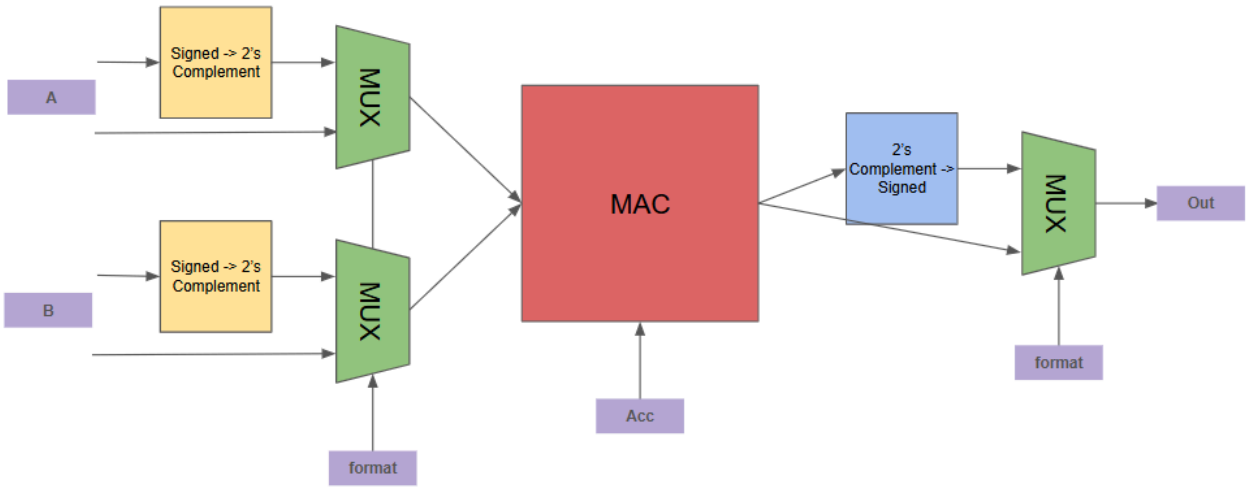
Reconfigurable MAC unit

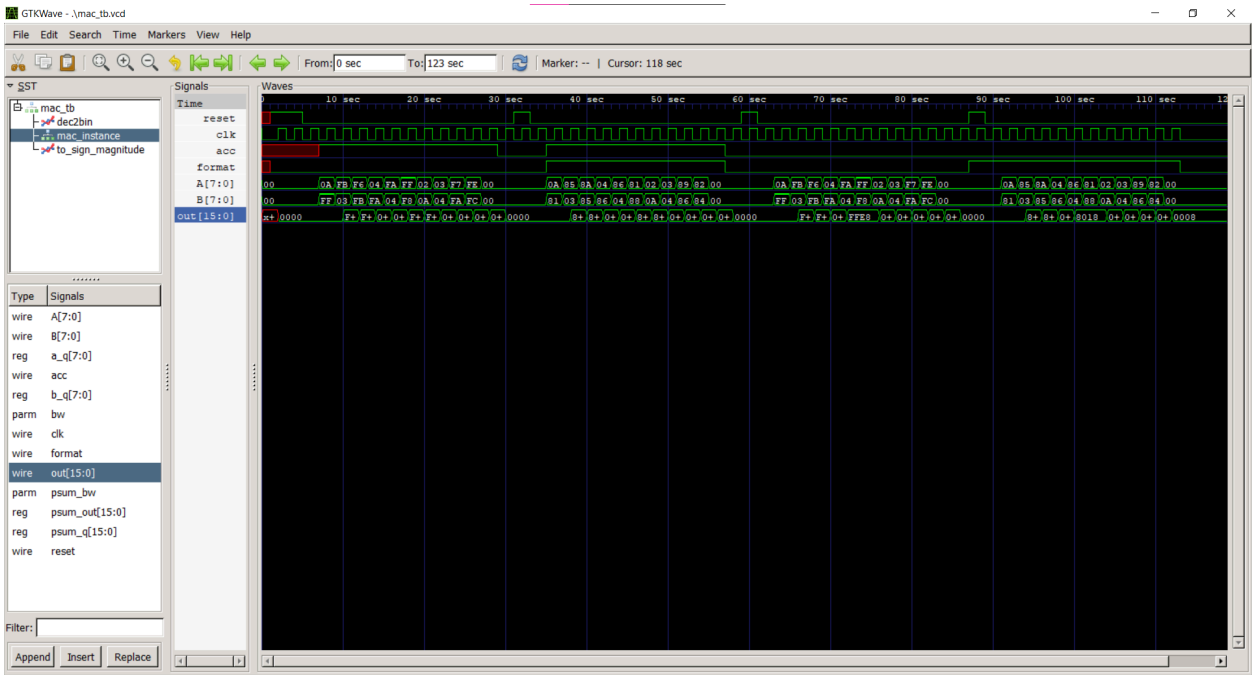
Name: Chaitany Raghav
TSN: 200478852

Architecture

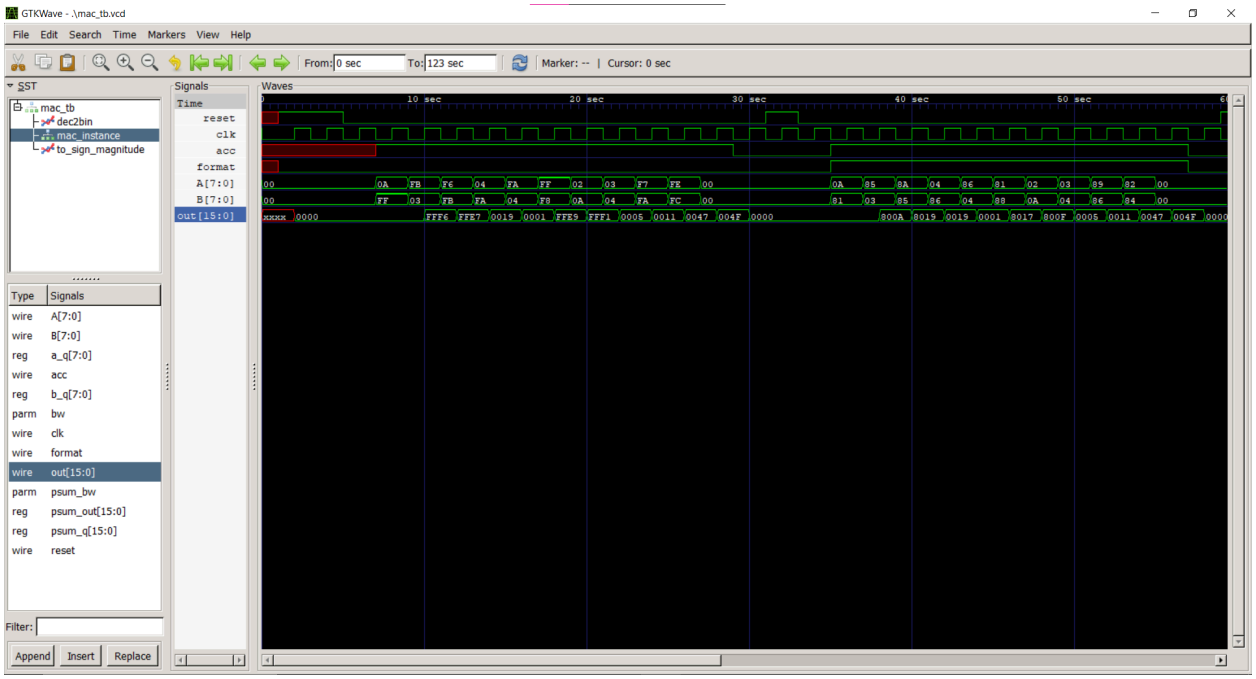


In my implementation, the MAC Unit works only with 2's Complement numbers; the input/output format is handled externally to the MAC logic.

Waveform 1: Complete



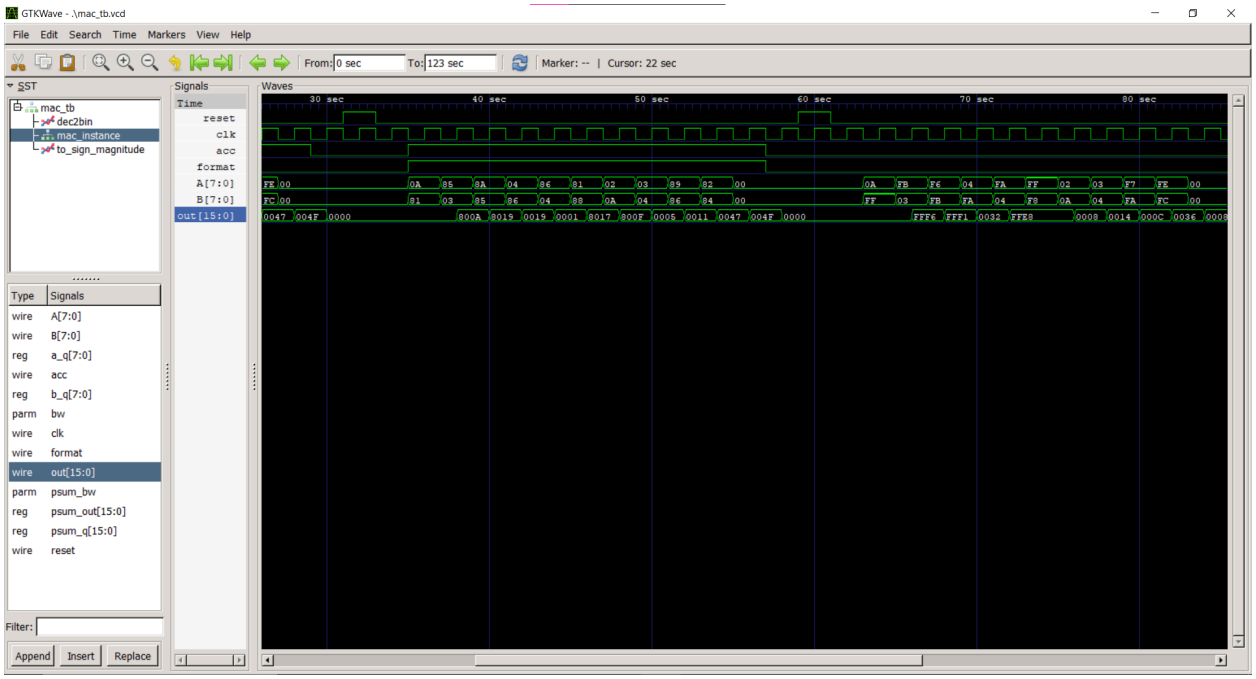
Waveform 2: acc = 1b’1, format = 1b’0



I/O Data:

A	B	Product	Running Sum	Output
10	-1	-10	-10	0xFFF6
-5	3	-15	-25	0xFFE7
-10	-5	+50	25	0x0019
4	-6	-24	1	0x0001
-6	4	-24	-23	0xFFE9
-1	-8	+8	-15	0xFFF1
2	10	+20	5	0x0005
3	4	+12	17	0x0011
-9	-6	+54	71	0x0047
-2	-4	+8	79	0x004F

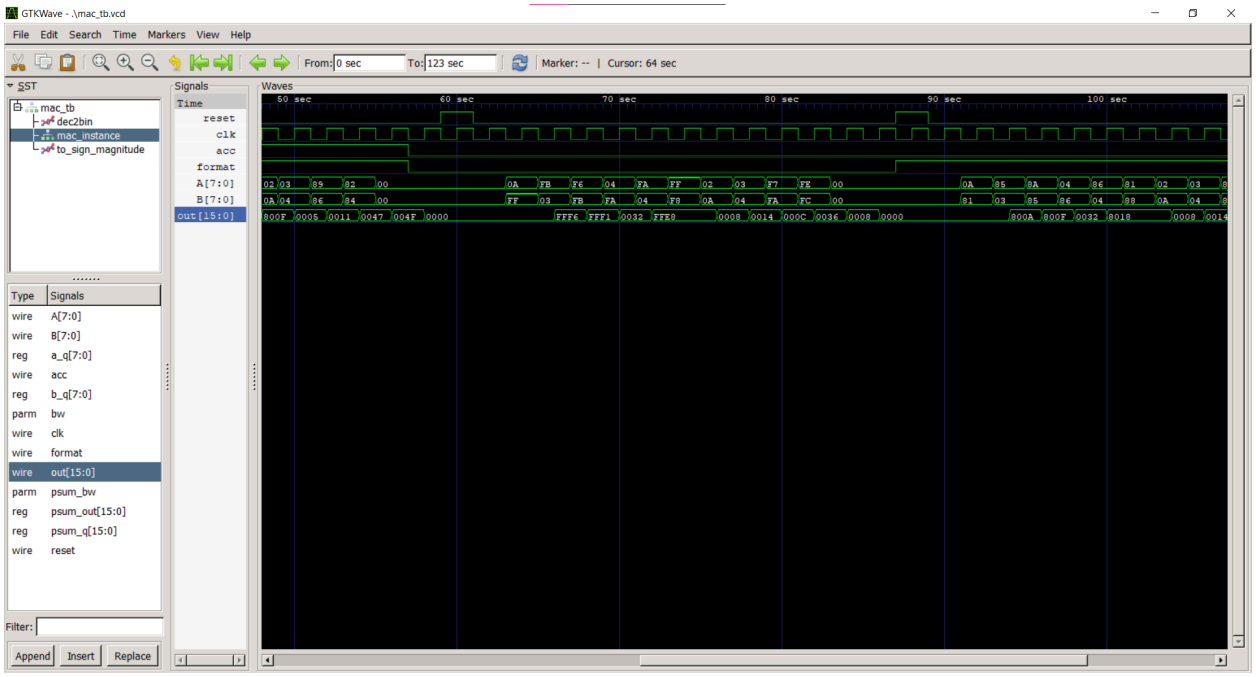
Waveform 3: acc = 1b’1, format = 1b’1



I/O Data:

A	B	Product	Running Sum	Output
10	-1	-10	-10	0x800A
-5	3	-15	-25	0x8019
-10	-5	+50	25	0x0019
4	-6	-24	1	0x0001
-6	4	-24	-23	0x8017
-1	-8	+8	-15	0x800F
2	10	+20	5	0x0005
3	4	+12	17	0x0011
-9	-6	+54	71	0x0047
-2	-4	+8	79	0x004F

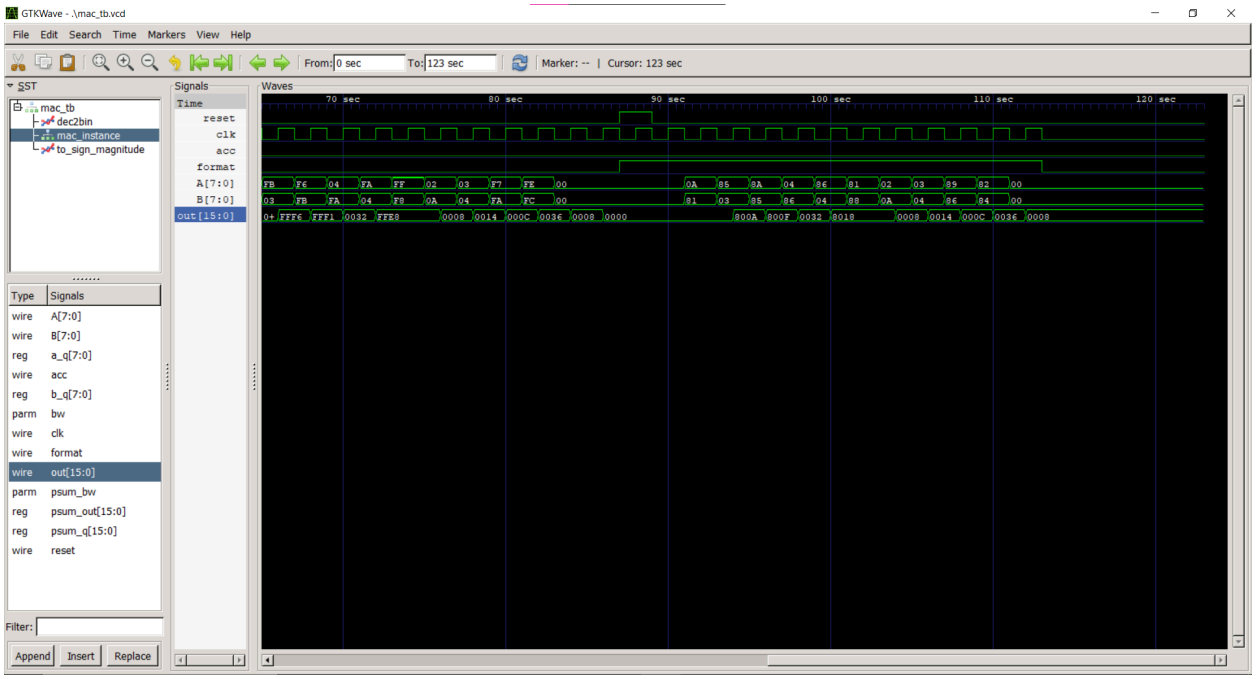
Waveform 4: acc = 1b’0, format = 1b’0



I/O Data:

A	B	Product	Output
10	-1	-10	0xFFF6
-5	3	-15	0xFFF1
-10	-5	+50	0x0032
4	-6	-24	0xFFE8
-6	4	-24	0xFFE8
-1	-8	+8	0x0008
2	10	+20	0x0014
3	4	+12	0x000C
-9	-6	+54	0x0036
-2	-4	+8	0x0008

Waveform 5: acc = 1b’0, format = 1b’1



I/O Data:

A	B	Product	Output
10	-1	-10	0x800A
-5	3	-15	0x800F
-10	-5	+50	0x0032
4	-6	-24	0x8018
-6	4	-24	0x8018
-1	-8	+8	0x0008
2	10	+20	0x0014
3	4	+12	0x000C
-9	-6	+54	0x0036
-2	-4	+8	0x0008

Code

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac (out, A, B, format, acc, clk, reset);

parameter bw = 8;
parameter psum_bw = 16;

input clk;
input acc;
input reset;
input format;

input signed [bw-1:0] A;
input signed [bw-1:0] B;

output signed [psum_bw-1:0] out;

reg signed [psum_bw-1:0] psum_q;
reg signed [bw-1:0] a_q;
reg signed [bw-1:0] b_q;

assign out = psum_q;

// Your code goes here

reg signed [psum_bw-1:0] psum_out;

// Input
always @(posedge clk) begin
    if (reset == 1'b1) begin
        a_q <= 0;
        b_q <= 0;
    end
    else begin
        if (format == 1'b0) begin
            // 2's complement
            a_q <= A;
            b_q <= B;
        end
```

```
        else begin
            // Signed magnitude
            a_q <= A[bw-1] ? -A[bw-2:0] : A[bw-2:0];
            b_q <= B[bw-1] ? -B[bw-2:0] : B[bw-2:0];
        end
    end
end

// MAC
always @(posedge clk) begin
    if (reset == 1'b1) begin
        psum_out <= 0;
    end
    else begin
        if (acc == 1'b0)
            psum_out <= a_q * b_q;
        else
            psum_out <= psum_out + a_q * b_q;
    end
end

// Convert internal result
always @(*) begin
    if (format == 1'b0) begin
        // 2's complement
        psum_q = psum_out;
    end
    else begin
        // Signed magnitude
        if (psum_out < 0)
            psum_q = {1'b1, -psum_out[psum_bw-2:0]};
        else
            psum_q = {1'b0, psum_out[psum_bw-2:0]};
    end
end

endmodule
```

UCSD Linux Cloud: N/A. This project was done on a local setup.
Github: https://github.com/chaitany-raghav/ece284fa26/tree/my-solutions